CHAPTER 1

INTRODUCTION

The rapid growth of the Internet of Things (IoT) has interconnected billions of devices across domains such as healthcare, smart homes, transportation, and industrial automation, enabling efficient data exchange but also creating major security and privacy challenges. Since IoT devices have limited processing power, memory, and energy, implementing strong yet lightweight security mechanisms are crucial. One of the biggest concerns is the risk of unauthorized access, data interception, and manipulation, which makes cryptography essential to ensure confidentiality, integrity, and authenticity of transmitted data. However, conventional cryptographic techniques are often too resource-intensive, highlighting the need for optimized algorithms designed specifically for low-power embedded environments.

This **"Secure Data Communication in IoT using Cryptography"** focuses on designing and integrating a hardware-backed lightweight cryptographic solution for IoT data protection. The objective is to implement efficient encryption techniques that safeguard communication between IoT devices and external systems, without significantly compromising performance or energy consumption. By using suitable symmetric or asymmetric cryptographic schemes, along with hardware/software optimizations, the project ensures that data is encrypted before transmission, making it resilient against common cyber threats such as eavesdropping, man-in- the-middle (MITM) attacks, and data breaches.

1.2 PROBLEM STATEMENT

Resource-constrained IoT Edge devices such as the ESP32 demand efficient real-time authenticated encryption (e.g., AES-GCM) for secure data communication. However, the lack of hardware-based secure key storage forces cryptographic keys to reside in host memory (RAM or flash), making them susceptible to physical and software-based extraction attacks, thereby weakening the overall security architecture and trust of the system.

1.3 OBJECTIVE

1. To design and realize a secure sensor data encryption mechanism on the ESP32 using AES-128 GCM.
2. To design and implement a secure key management mechanism using the ATECC608A for master key protection and session key derivation.
3. To design and validate end-to-end secure communication between the ESP32 and server.

1.4 Relevancy of the SKILL TRAINING Academic courses in project Selection

We completed a course on NPTEL titled Industrial IoT, through which we gained foundational knowledge about IoT systems, communication protocols, and real-world applications. Additionally, we undertook a course on Coursera focused on cryptography, where we learned essential concepts such as data encryption, decryption, and secure communication techniques. The knowledge gained from these courses played a significant role in selecting and shaping our project titled **"Secure Data Communication in IoT using Cryptography."** These learnings helped us understand how to protect IoT devices and data from unauthorized access, ensuring secure and reliable communication within IoT networks.

1.5 Overview of Report

This project is about making communication between IoT devices more secure. Nowadays, many devices like sensors and smart systems are connected to the internet, but they are not very powerful.

Because of this, they cannot handle heavy security methods, which makes them easy targets for attacks.

In our project, we tried to solve this problem by using a simple and efficient security method. We used an ESP32 microcontroller to collect data from a sensor (like temperature and humidity). Before sending this data over the internet, we encrypt it so that no one else can read or change it.

To make the system more secure, we used a special hardware chip called ATECC608A. This chip safely stores secret keys and helps in generating secure keys for encryption. The important point is that the main key is never exposed, which makes the system safer even if someone tries to hack the device.

The data is encrypted using AES-128 GCM, which not only protects the data but also checks if the data is modified during transmission. Then the encrypted data is sent to a server through Wi-Fi. On the server side, the data is verified and decrypted back to its original form.

We tested the system in different conditions. When normal data is sent, it is received correctly. If someone tries to change the data, the system detects it and rejects it. It also prevents replay attacks by ensuring each data transmission is unique.

Overall, this project shows that we can achieve secure communication in IoT devices using light weight cryptography and hardware support without affecting performance too much.

CHAPTER 2

METHODOLOGY AND PROPOSED WORK

The proposed system securely collects environmental sensor data using an ESP32 MCU, as illustrated in **Figure 2.1**. The ESP32 receives plaintext readings from the DHT11 sensor and encrypts them using AES-128 GCM. The Master Key (Km) is generated through an internal True Random Number Generator (TRNG) and remains permanently isolated within the Microchip ATECC608A secure element. During operation, the ESP32 requests an encryption key, prompting the ATECC608A to derive an ephemeral Session Key (Ks), which is temporarily loaded into the ESP32 hardware accelerator for encryption. The encrypted payload, consisting of the ciphertext, Initialization Vector (IV), and MAC tag, is then transmitted as a packet over Wi-Fi to the server, as shown in **Figure 2.1**. On the server side, a matching session key is independently derived from a shared master secret to verify the MAC tag for integrity and decrypt the data for secure display.

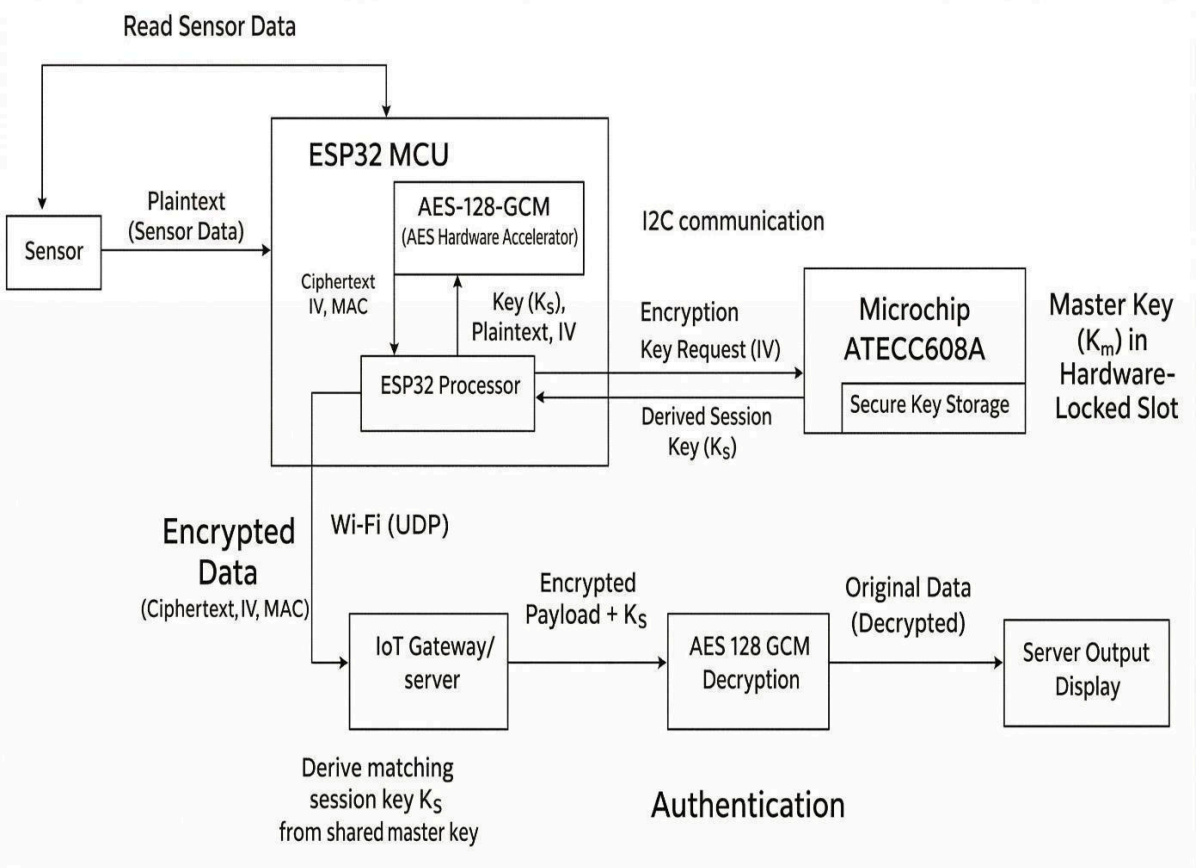

Figure 2.1: Block diagram of Secure Data Communication

2.1 Sensor Data Acquisition

- ESP32 requests data from the sensor.
- Sensor sends plaintext sensor values to ESP32.

2.2 Secure Key Management (ATECC608A)

- A The Master Key (Km) is generated by the ESP32 TRNG during a one-time provisioning phase and is then securely injected and hardware-locked within the ATECC608A.
- The Master Secret is securely locked inside the ATECC608A and cannot be exported or read by external software.
- The ATECC608A derives a unique, ephemeral Session Key (Ks) for each encryption cycle, ensuring perfect forward secrecy for every transmitted packet

2.3 Encryption on ESP32

- ESP32 hardware accelerator performs AES-128 encryption, while GCM-based authentication (GHASH) is handled in software to achieve authenticated encryption.
- ESP32 TRNG generates a unique Initialization Vector (IV) for every session to ensure cryptographic freshness.
- ATECC608A provides the derived Session Key (Ks) to the ESP32 for the encryption operation.
- Output generation produces the Ciphertext and Authentication Tag (MAC) for secure transmission.

2.4 Data Transmission

- ESP32 sends encrypted data to the IoT server via Wi-Fi.

2.5 Server Processing

- Server receives the encrypted packet (Ciphertext, IV, and Authentication Tag)
- Server derives the matching Session Key (Ks) using the shared Master Secret.
- Verifies the MAC to ensure data integrity and authenticity.
- Decrypts the ciphertext to obtain the original data.

2.6 Final Output

- Server displays the decrypted, original sensor values.

2.7 FLOWCHART:

The secure data handling process illustrated in **Figure 2.2** begins with the ESP32 receiving plaintext sensor data from the DHT11 sensor and deriving an ephemeral session key (Ks) from the ATECC608A secure element, where the master secret is securely stored. Using this session key, the ESP32 performs AES-128-GCM encryption to generate ciphertext and an authentication tag. The encrypted data, along with the Initialization Vector (IV), is then transmitted over Wi-Fi to the receiver, as shown in **Figure 2.2** On the receiver side, the backend system derives the matching session key, verifies the authentication tag for integrity, and decrypts the ciphertext to securely retrieve the original sensor data.

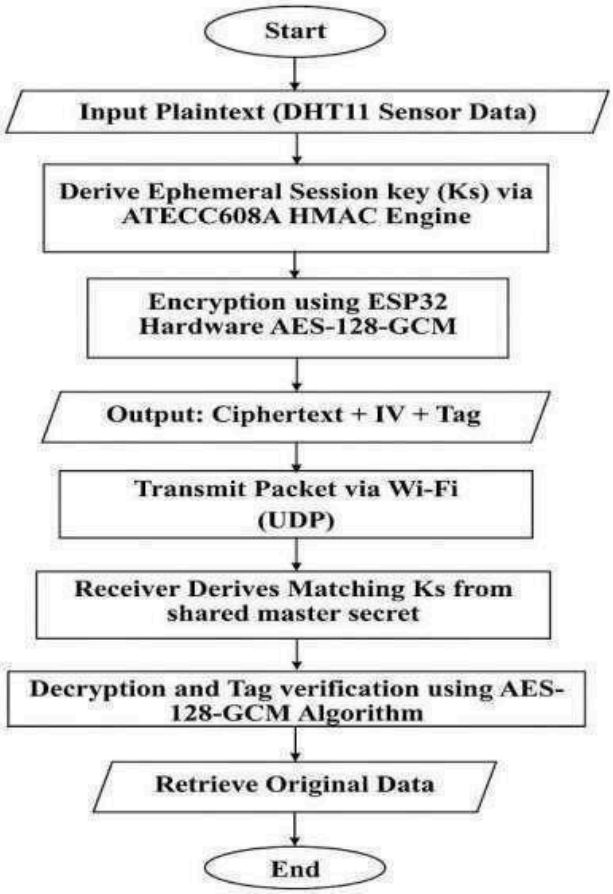

Figure2.2: Block diagram of Secure Data Communication

WORKINGMECHANISM: Hybrid Secured Data Communication

2.8 Plaintext Data Generation(IoT Edge)

The IoT device (ESP32) functions as the Edge Node, acquiring live plaintext sensor readings (e.g., temperature and humidity via DHT11). Before transmitting to the server, these readings must be encrypted to prevent eavesdropping and man-in-the-middle attacks.

2.9 Key Management: Secure Storage and Session Key Derivation

This stage establishes a shared symmetric secret between sender and receiver using the Microchip ATECC608A Secure Element.

2.9.1 Key Generationand Storage (Inside ATECC608A)

Secure Provisioning: The Master Key (Km) is generated by the ESP32's internal TRNG during a one-time provisioning phase and injected into the ATECC608A.

- The key is stored in a dedicated Secure EEPROM slot within the ATECC608A, which is permanently hardware-locked to prevent any future external readout.
- Once locked, the Master Key (Km) remains isolated within the secure element; it is never visible to the ESP32's software memory, effectively preventing physical or side- channel extraction.

2.9.2 EphemeralSessionKey Derivation(Runtime Phase)

Instead of using a static key for every transmission, the system derives a unique session key (Ks) to limit data exposure:

- ESP32 initiates a request to the ATECC608Avia I2C to derive a new key.
- The ATECC608A uses its internal HMAC engine and the Master Secret to compute a 128- bit Session Key (Ks).
- This derivation process is analogous to "Perfect Forward Secrecy," where even if one session key is compromised, the Master Secret remains safe.
- The server-side Python environment utilizes the same Master Secret to derive the identical matching Session Key.

2.9.3 ControlledData Flow for Encryption

- The derived Session Key is transferred to the ESP32 internal memory only during the active encryption window.
- The key is loaded into the ESP32 AES accelerator registers and is cleared from volatile memory immediately after the ciphertext is generated.

2.10 Encryption (Sender Side – ESP32 Hardware Accelerator)

The encryption stage transforms plaintext sensor data into AES-GCM authenticated ciphertext. The ESP32 generates a unique Initialization Vector (IV) per transmission to ensure cryptographic uniqueness.

- The Session Key derived from ATECC608A is loaded into the AES hardware registers.
- ESP32 hardware accelerator performs AES-128 encryption, while the GCM authentication process is implemented in software to ensure data integrity and authenticity on the DHT11 sensor values.
- Encryption outputstwo data components:
  - Authentication Tag (MAC) ensuring packet integrity and authenticity.
  - Ciphertext (confidential payload)

2.11 Transmission of Secured Packet

The ESP32 assembles a **Structured Binary Frame** by concatenating the cryptographic outputs into a single contiguous memory buffer.

The packet structure follows a strict **Byte-Offset Protocol**:

- **Header(IV):** The first 32 bytes contain the Initialization Vector.
- **Authentication (Tag):** The next 16 bytes contain the GCM Authentication Tag (MAC).
- **Payload (Ciphertext):** The remaining bytes contain the encrypted sensor data.

This **Binary Payload** is transmitted to the IoT server via Wi-Fi using the **User Datagram Protocol (UDP)**. This protocol is selected to eliminate the overhead of text-based formatting and TCP handshakes, ensuring minimal communication latency and maximum throughput real.

2.12 Decryption and Verification (Receiver Side – Python Server)

- The server possesses the shared Master Secret (Km) and derives the matching 128-bit Session Key.
- The receiver performs Tag verification by recomputing theMAC using:
    - Received Ciphertext
    - Received IV
    - Derived Session Key
- If verification succeeds, the ciphertext is decrypted to recover the original sensor readings.
- Modified or corrupted packets are automatically rejected since a Tag mismatch exposes tampering.

2.13 Integrity and Security Proof

- Confidentiality: Plaintext is never transmitted, preventing eavesdropping on the Wi-Fi network.
- Integrity: The AES-GCM authentication tag prevents unauthorized modification of sensor data.
- Key-at-Rest Security: The Master Secret is securely locked inside the ATECC608A and can never be read in plaintext by the ESP32.
- Session Isolation: Using derived ephemeral keys ensures that the long-term Master Secret is never directly used for encryption.
- Replay Attack Defence: Because a unique IV and Session Key are used for every transmission, an attacker cannot "replay" a recorded message to fool the server.

CHAPTER 3

COMPONENTS REQUIRED

3.1 ESP32 Development Borad

The ESP32, shown in **Figure 3.1**, serves as the primary microcontroller responsible for data acquisition, encryption, and wireless transmission.

It includes:

- Dual-core Tensilica LX6 processor
- Built-in Wi-Fi and Bluetooth
- Hardware-accelerated AES, SHA, and RNG modules

Figure 3.1: ESP32 Microcontroller

3.2 Sensors (DHT11 / DHT22 / BMP280 / any required sensor)

The system is designed to be compatible with various sensors (such as DHT22 or BMP280); however, for this implementation, a **DHT11 temperature–humidity sensor (Figure 3.2)** is interfaced with the ESP32 to generate **real-time plaintext data**.

The sensor provides two environmental measurements:

- **Temperature** (°C)
- **Humidity** (%)

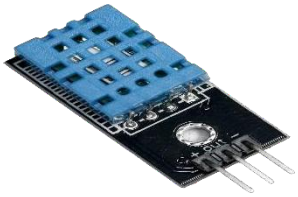

Figure 3 .2: DHT11 Sensor Module

3.3 MicrochipATECC608ASecure Element

The ATECC608A, shown in **Figure 3.3**, is a dedicated cryptographic co-processor that provides a Hardware Root of Trust for IoT devices. It isolates sensitive cryptographic operations from the ESP32 and ensures that private keys are never exposed in the host microcontroller's memory.

- EAL 4+ certified secure hardware
- Supports ECC, AES-128, SHA-256, HM

Figure 3.3: ATECC608A Breakout Board

3.4 Software Tool

The required software components for this project are shown in the table 3.1

Table3.1: Required Software Components

Tool/Library	Specification	Why It's Needed	How It Is Used
ESP-IDF Framework	**Development environment and compiler toolchain for the ESP32.**	Base Environment: Provides the drivers, Wi-Fi stack, and the low-levelfunctions to access the ESP32 hardware.	Used to write, compile, and upload the C/C++ firmware to the ESP32**.**
Mbed TLS AESLibrary	**Standardize d, open-source cryptograph i c library.**	Crypto API: Provides the function calls and logic for the AES-128-GCM authenticated encryption scheme.	Configured to leverage the ESP32's native hardware module for maximum performance.
Microchip Crypto Auth Lib	**Crypto Auth Lib Specific driver library for the ATECC608A**	Key Management Interface: Enables the ESP32 to communicate withthesecureelement via I²C.	Used for Key Provisioning and initiating the session key derivation (Ks) within the hardware vault.
Python 3.x / VS code	**Server-side runtime environment .**	Backend Processing: Handles the receptionand decryption of IoT data packets.	Used to run the server script that verifies the MAC tag and decrypts the sensor data.

Chapter 4

DESIGN AND IMPLEMENTATION

This chapter presents the stages of the integration of the hardware security element, the microcontroller, and the backend server. The implementation combines the security requirements and the theory of the IoT ecosystem into a communication system.

4.1 Hardware Setup and Initialization

The journey began with the configuration of the hardware setup of the I2C communication of the ESP32 and the Microchip ATECC608A secure element.

- **Bus Scan and Wake up:** The I2C bus was built with GPIO 21 as SDA, and GPIO 22 as SCL. The tests ran successfully confirming ATECC608A at address 0x60 or 0xC0 (the address shifted). The ATECC608A is a low-power micro-device that sleeps to save power. The SDA line was brought low for > 60 microseconds to enable wake up before I2C commands.
- **Diagnostic Verification:** Before a security policy was implemented. The system performed a diagnostic read of the unique 9-byte serial number of the chip and confirmed the I/O was operational.

4.2 The Cryptographic Provisioning Phase

The most essential and important phase of the implementation was the provisioning of the cryptographic keys. To ensure the keys were physically protected from adversaries, a hardware isolation strategy was adopted.

- **Configuration Zone Sealing:** The configuration zone of the ATECC608A was irreversibly sealed. This act to engage security policies and slot definitions to ensure they could not be changed.
- **IO Protection Key Setup:** To protect the I2C bus from any sniffing (unauthorized observation) attacks during the setup phase, a 32-byte IO Protection Key was generated by the ESP32's True Random Number Generator (TRNG) during the configuration phase and was deposited into Slot 6 of the secure element.
- **Encrypted Master Key Injection:** The primary Master Key (Km) was produced with the

TRNG. Instead of transmitting this key unencrypted on the I2C bus, an encrypted write operation on Slot 5 was achieved using the procedure atcab_write_enc to write into Slot 5, protected by the previously set IO Protection Key

- **Data Zone Sealing:** After the Master Key was confirmed, with a MAC challenge, the entire Data Zone was locked, thus the key was rendered physically non-exportable.

4.3 Runtime Operation and Data Transmission

After establishing the Root of Trust, a runtime application was designed to gather, encrypt ,and transmit data on the fly.

- **Sensor Interfacing:** The ESP32 interfaces with the DHT11 sensor, providing simple access to real-time temperature and humidity data. This data is read directly from the sensor by the ESP32 by connecting and accessing GPIO 4. Once the data is obtained, the ESP32 formats this data into a readable string.
- **Ephemeral Session Key Derivation:** The system is designed with a philosophy of Perfect Forward Secrecy and will not directly encrypt data using the core Master Key. Instead, the ESP32 synthesizes a 32 byte random Initialization Vector (IV) and subsequently, the ATECC608A uses its embedded HMAC SHA256 to create a Session Key (Ks) by combining the IV and the Master Key stored in Slot 5.
- **Hardware AES-GCM Encryption:** The mbedtls library on the ESP32 utilizes an AES hardware accelerator, such that the Session Key is derived, and the sensor payload is encrypted, producing both ciphertext and a 16 byte Authentication Tag.
- **UDP Wi-Fi Transmission:** The ESP32 assembles a binary packet where the IV is 32 bytes, the Tag is 16 bytes, and the remaining bytes are ciphertext. These UDP packets are transmitted over wifi via UDP socket to reach the exact destination server port 8080.

4.4 Mathematical Models and Cryptographic Algorithms

We integrate stable cryptographic methods into the system's core security model. Mathematical theories and operations safeguard and authenticate information in transmission over the network.

1.Ephemeral Session Key Derivation (HMAC-SHA256)

Rather than employing a static master key (Km) to authorize the transfer of information, the ATECC608A, generates a new unique session key, (Ks), specifically for each transaction. The process incorporates the use of a Hash based Message Authentication function (HAMC).

The mathematical function is as follows:

Ks = HMAC-SHA256(Km,IV)

Where IV (Initialization Vector) is a nonce, the dynamic and cryptographically strong salt. Because IV dynamically changes for each and every packet, a unique 128-bit session key is generated for every transmission ensuring Perfect Forward Secrecy.

2. AES-128 GCM (Galois/Counter Mode) Architecture

The session key generated is used by the ESP32 to encrypt the payload that is sensor data. GCM was the operating mode of choice, as it is capable of providing cryptography wherein both encryption and a mathematical proof of integrity (i.e., privacy and authenticity) of the data is assured. Two fundamental processes make it possible for the GCM operating mode to provide AEAD, both are capable of operating simultaneously:-

Confidentiality (AES-CTR): AES in Counter mode, takes as input the plaintext (P) and encryption key (Ks) and outputs ciphertext (C) wherein Ks is a session key and IV, and C is the resultant of the exclusive OR (XOR) operation.

$$C = P \oplus \text{AES}_{K_s}(\text{Counter})$$

- **Integrity (GHASH)**: An integrity proof (i.e., a unique Authentication Tag (T)) is produced via the Galois field multiplication (GF(2^{128}). The output of the cipher is then input to the GHASH function, along with a subkey (H), to produce a 16-byte MAC tag. A single modification of even one bit of the ciphertext by an **attacker** during Wi-Fi transmission, the GHASH which is independently computed by the receiver will differ, and will be determined to be in contradiction to the integrity of the output, then causing the packet to be immediately dropped.

$$T = \text{GHASH}_H(C)$$

4.5 Hardware Timing Constraints and Protocol Specifications

Stable communication between the ESP32 and the ATECC608A hinges on the following hardware timing constraints for the I2C bus.

- **Baud Rate(I2C Bus):** The I2C interface uses a standard baud rate of 100 kHz. The ATECC608A supports up to 1 MHz, however, due to the high capacitance on a breadboard/prototyping environment, 100 kHz was chosen to better signal integrity.

- **Chip Sleep State and Manual wake up:** The ATECC608A deep sleeps to save power. A standard I2C start condition cannot wake the chip. Therefore, a manual wake-up is performed by pulling SDA to logic LOW for 80 microseconds. The 80 microseconds is above the hardware requirement of 60 microseconds.
- **Wake up Delay:** After the wake pulse, the bus needs to be idle for the time it takes to the CPU cryptographic co-processor to bring its internal state. A programmatic delay of 1.5 ms is strictly enforced before the ESP32 sends the first I2C command.

4.6 Resource Utilization and Architecture Trade-offs

One of the main tenets of IoT cryptography is finding the equilibrium between high-value security and the restricted resources.

- **Hardware Acceleration over Software Execution:** The system leverages the mbedtls cryptographic library which integrates with the ESP32's hardware AES accelerator. Should AES-128 be performed entirely in software it would use a large number of CPU cycles, prevent the CPU from gathering sensor data and consume a large amount of power consequently. The encryption operations are performed in silicon. The CPU is freed for network stack management.
- **Memory Footprint Optimization:** Significant RAM is required for storing complex arrays and performing complex multiplications. The ESP32 is spared the volatile memory usage due to the ATECC608A performing HMAC SHA256 operations inside the chip. The Session Key is transferred temporarily to the AES hardware registers and is erased immediately after the packet is created. This minimizes memory usage and renders cold-boot or RAM dumping attacks ineffective.

4.7 Receiver Backend and Security Validation

The backend architecture is created using a Python-based server ecosystem. This server is the IoT gateway to securely receive and process (parse and authenticate) and log the telemetry data for the ESP32 edge nodes.

4.7.1 UDP Socket Initialization and Packet Parsing

To offer a low latency service, the server skips creating a connected socket and listens for UDP frames (broadcasted via the User Datagram Protocol) instead.

- **Network Binding:** A UDP socket (SOCK_DGRAM) bound to port 8080 and listens to

all the available network interfaces (0.0.0.0).

- **Byte-Level Slicing:** Whenever a UDP payload is received in the 1024 bytes socket receive buffer, the server must slice the structured, binary frame before the cryptographic operations. The byte array shall be parsed strictly per defined protocol.
- **Bytes [0:32]:** will be the IV of the encryption frame.
- **Bytes [32:48]:** will be the 16 byte GCM Authentication Tag.
- **Bytes [48:end]:** will be the Ciphertext.

4.7.2 Symmetric Key Derivation (Server-Side)

As there is the utilization of the ephemeral keys in this system, the server must compute the needed Session Key (Ks) for each incoming packet. The server performs an HMAC-SHA256 operation of the 32 byte IV alongside the Master Key, which is securely stored on the server. The first 16 bytes of the resulting digest is then taken to constitute the AES-128 Session Key, mirroring the hardware derivation occurring inside the ATECC608A.

4.7.3 Decryption, Authentication, and Error Handling

The backend cryptography engine uses the Crypto. Cipher AES library, operating in GCM mode.

- **Nonce Extraction:** The leading first 12 bytes of the received 32-byte Initialization Vector (IV) work as a standard GCM nonce.
- **AEAD Verification:** The decrypt_and_verify method is run in parallel to decrypt the cipher and verify the Authentication Tag.
- **Exception Handling:** In the event of a man-in-the middle alteration of the cipher text, the GHASH computed during the serve will be unlike the Tag received. This triggers a Value Error exception in the python environment, and the server will immediately terminate the session.

4.7.4. Replay Attack Mitigation Architecture

Replay Attack is one of the most severe attacks on the security of wireless IoT faced in most wireless networks. This type of an attack will involve the adversary sending the same mechanism of the system. The server attempts to address this problem by implementing a form of packet tracking mechanism.

- **Tracking Mechanism:** The python backend maintains a seen ivs set attribute in the memory.
- **Verification Logic:** The server first checks if the IV of the incoming packet is already in the set, and if so, the packet will not be processed as sets allow for a very efficient **O (1)** lookup. This effectively has a virtual latency of 0 to the server.
- **Attack Rejection:** All packets containing duplicate IVs are instantly declared as replay attack packets. A security alert is triggered, and all subsequent processing is ceased. Running several tests with your simulated attacker scripts, we were able to successfully and efficiently block all attempts to aggressively and rapidly replay packets.

4.8 CIRCUIT EXPLANATION

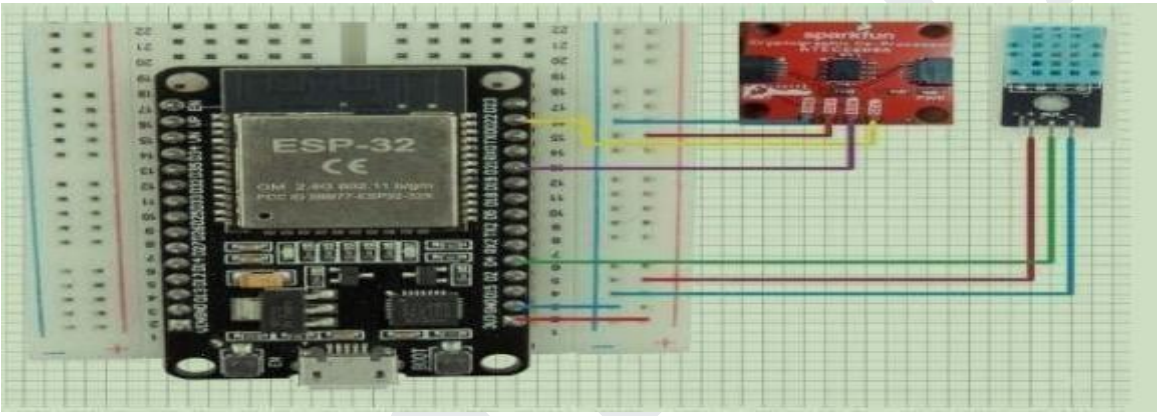

Figure 4.1: circuit connection image

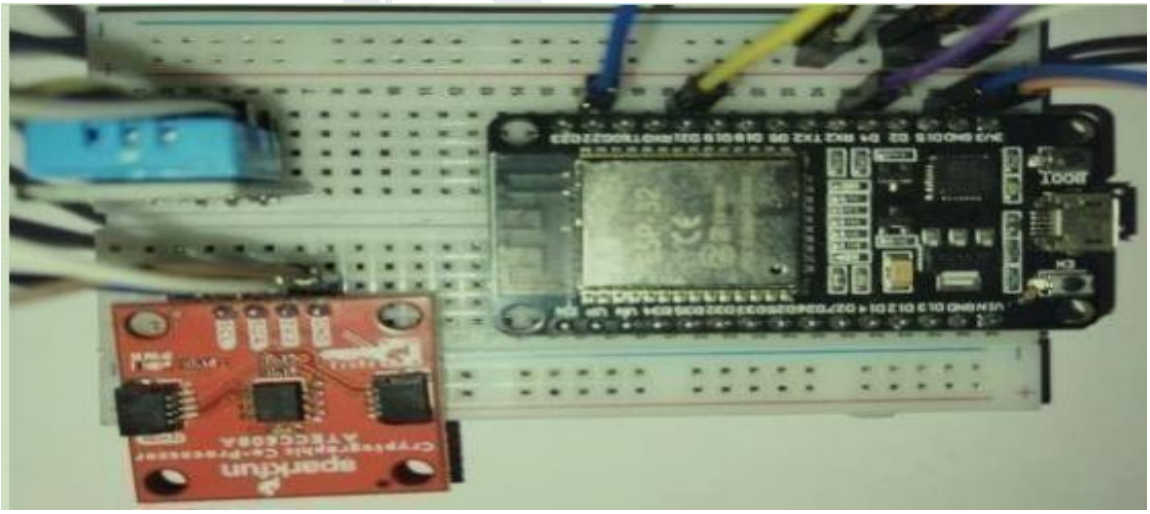

Figure 4.2: Overview of circuit connection image

Hardware Connections

The ATECC608A secure element, shown in **Figure 4.1** and **Figure 4.2**, communicates with the ESP32 using the I2C protocol. The VCC pin of the ATECC608A is connected to the 3.3V supply of the ESP32, while the GND pin is connected to the common ground. For communication, the SDA pin is connected to GPIO 21 of the ESP32, and the SCL pin is connected to GPIO 22. This setup enables efficient and secure data transfer between the ESP32 and the cryptographic chip.

For sensing environmental data, the DHT11 sensor is interfaced with the ESP32. The VCC pin of the DHT11 is connected to either 3.3V or 5V, although 3.3V is preferred to maintain compatible logic levels with the ESP32. The data (output) pin of the sensor is connected to GPIO 4 of the ESP32, which operates using a one-wire serial communication interface. The GND pin is connected to the common ground, ensuring proper circuit operation. This configuration allows the ESP32 to collect real-time temperature and humidity data from the environment.

Chapter 5

RESULTS AND DISCUSSION

5.1 Results:

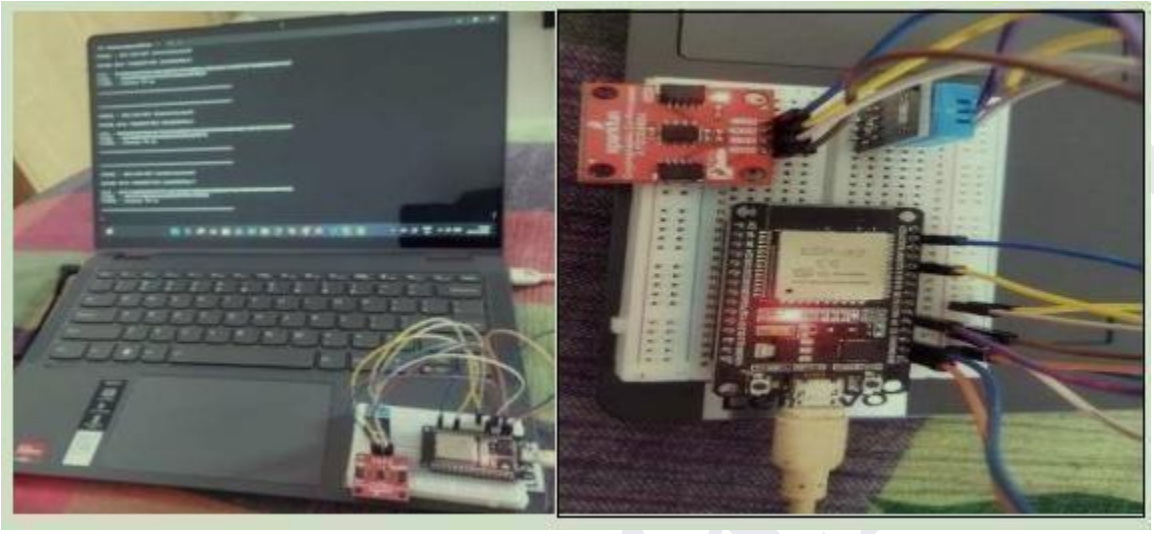

Figure 5.1: Output Images

```
I (4249) wifi:state: assoc -> run (0x10)
.I (4299) wifi:connected with POCO M3, aid = 7, channel 6, BW20, bssid = 7e:dc:2a:d3:d9:88
I (4299) wifi:security: WPA2-PSK, phy: bgn, rssi: -38
I (4309) wifi:pm start, type: 1

I (4309) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (4309) wifi:dp: 2, bi: 102400, li: 4, scale listen interval from 307200 us to 409600 us
I (4319) wifi:AP's beacon interval = 102400 us, DTIM period = 2
..I (5359) esp_netif_handlers: sta ip: 10.188.38.223, mask: 255.255.255.0, gw: 10.188.38.146
.
WiFi Connected.

================================================

[MODE] : AES-128-GCM (Authenticated)

SECURE DATA TRANSMITTED SUCCESSFULLY

Sensor Data : Temperature:32.0C,89.6F | Humidity:52.0%

[IV] : D36DB2A14814B4D1A8444BFBB3087EE4797236FF8B653D38F1DFC8F372CDB6A5
[TAG] : 6A8CCA607D36B0DEB71B351A0D8535BB
Encrypted Data : 02A5DF14EB807491DB259FC127D89300B3332044404949494A5B07B7841774E1F1A7492BA437F7DA08
[TIME] : Latency 72 ms

================================================
```

Figure 5.2 Output Image of Encrypted Data transmitting from ESP32

```
def start_listener():
    try:
        cipher = AES.new(session_key, AES.MODE_GCM, nonce=received_iv[:12])

        # If this line succeeds, both key and MAC are valid
        decrypted_data = cipher.decrypt_and_verify(ciphertext, received_tag)

        print(f"SESSION KEY STATUS  : CORRECT (Generated Using Master Secret)")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[Running] python -u "c:\Users\suman\Documents\pythoncoding\secure_reciever_server.py"
Listening for secure IoT data on port 8080...

============================================================
SOURCE ADDRESS       : 10.188.38.223:52584
ENCRYPTED PAYLOAD    : 02A5DF14EB807491DB259FC127D89300B33320444049494A5B07B7841774E1F1A7492BA437F7DA08
RECEIVED MAC (TAG)   : 6A8CCA607D36B0DEB71B351A0D8535BB
SESSION KEY STATUS   : CORRECT (Generated Using Master Secret)
MAC VERIFICATION     : SUCCESS (Integrity Verified)
DECRYPTED DATA       : Temperature:32.0C,89.6F | Humidity:52.0%
============================================================
```

Figure 5.3: Output Image of successfully Received, Verified and Decrypted the IoT sensor data

```
def start_listener():
    try:
        print(f"SESSION KEY STATUS  : CORRECT (Generated Using Master Secret)")
        print(f"MAC VERIFICATION    : SUCCESS (Integrity Verified)")
        print(f"DECRYPTED DATA      : {decrypted_data.decode('utf-8')}")

    except ValueError:
        # This specific error triggers if decrypt_and_verify fails the MAC check
        print(f"SESSION KEY STATUS  : UNKNOWN (Potentially correct)")
        print(f"MAC VERIFICATION    : FAILED! (Data tampered or wrong Master Key)")
```

```
[Running] python -u "c:\Users\suman\Documents\pythoncoding\secure_wifi.py"
Listening for secure IoT data on port 8080...

============================================================
SOURCE ADDRESS       : 10.114.140.223:63192
ENCRYPTED PAYLOAD    : 3CA8603A22C3F6DC76B2F57FD23284963301623CC3379625BE13CBEF0FB27156876FA90A2645872A
RECEIVED MAC (TAG)   : 6A39A10F1830DC3D04A9399DB7660CDD
SESSION KEY STATUS   : UNKNOWN (Potentially correct)
MAC VERIFICATION     : FAILED! (Data tampered or wrong Master Key)
DECRYPTED DATA       : [REDACTED/UNAVAILABLE]
============================================================

============================================================
SOURCE ADDRESS       : 10.114.140.223:63192
ENCRYPTED PAYLOAD    : 819B728AB02D1589B583AB49581EED8A3A3F9DDB7784E7EC2049899A55D64F1A0E529579EABF2E58
RECEIVED MAC (TAG)   : FCB7B1D2D7E482BBB7AD8A7DA17850DC
SESSION KEY STATUS   : UNKNOWN (Potentially correct)
MAC VERIFICATION     : FAILED! (Data tampered or wrong Master Key)
DECRYPTED DATA       : [REDACTED/UNAVAILABLE]
============================================================
```

Figure 5.4: Output Image of Received Data Tampered Then data packet rejected

```
# 1. Extract parts
received_iv = data[:32]
received_tag = data[32:48]
ciphertext = data[48:]

# 2. To Stop Replay Attacks
if received_iv in seen_ivs:
    print(f">>> SECURITY ALERT: Replay Attack Detected! Duplicate IV rejected.")
    continue
seen_ivs.add(received_iv)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[Running] python -u "c:\Users\suman\Documents\pythoncoding\a1.py"
Listening for secure IoT data on port 8080...

============================================================
SOURCE ADDRESS       : 192.168.29.208:52764
ENCRYPTED PAYLOAD    : A4A91DE0B43001BD98210C756AF1236617B327052EDEF5A7AA7039FE6BF5663F5E314D1EE9A48C27
RECEIVED MAC (TAG)   : 14B89FD7DA09BBEA37D8F71249750B71
SESSION KEY STATUS   : CORRECT (Generated Using Master Secret)
MAC VERIFICATION     : SUCCESS (Integrity Verified)
DECRYPTED DATA       : Temperature:32.0C,89.6F | Humidity:51.0%
============================================================

>>> SECURITY ALERT: Replay Attack Detected! Duplicate IV rejected.
>>> SECURITY ALERT: Replay Attack Detected! Duplicate IV rejected.
>>> SECURITY ALERT: Replay Attack Detected! Duplicate IV rejected.
>>> SECURITY ALERT: Replay Attack Detected! Duplicate IV rejected.
```

Figure 5.5: Replay Attack Test

The proposed system for secure data communication in IoT was successfully implemented and tested. The ESP32 collects real-time sensor data from the DHT11 and encrypts it using cryptographic techniques such as AES along with authentication mechanisms like HMAC. The ATECC608A ensures secure key storage and enhances the overall security of the system.

The encrypted data was successfully transmitted from the ESP32, as observed in **Figure 5.2**, where the data appeared in ciphertext form, ensuring confidentiality. At the receiver side, as shown in **Figure 5.3**, the data was successfully received, verified, and decrypted. The integrity and authenticity of the data were confirmed before decryption, and the original sensor values were accurately retrieved, proving reliable and secure communication.

Furthermore, as illustrated in **Figure 5.4**, the system effectively detected tampered data and rejected invalid packets. We successfully prevented a Replay Attack by identifying and discarding repeated or outdated data transmissions and observed in **Figure 5.5** . This demonstrates the robustness of the system against common security threats.

Overall, the results confirm that the project successfully achieved secure data transmission with confidentiality, integrity, and authentication, while also providing protection against tampering and replay attacks. Thus, the system works as intended and validates the effectiveness of implementing cryptography in IoT-based communication.

5.2 OBSERVATION TABLE:

Table 5.1: Observation Table

SL.NO	Test Condition	Input Given	System Response
1	Standard Operation	Valid DHT11 Data	Integrity verified, Data Decrypted
2	Tampering Attack	Altered Ciphertext	MAC verification failed, Data rejected
3	Key Isolation	Master Key Read Attempt	Kept isolated inside ATECC608A
4	Replay Attack	Duplicate IV Transmitted	Attack prevented via unique IV

1.Standard Operation:

Input: Valid DHT11Data

System Response: Integrity verified, Data Decrypted

Explanation: In normal operation, the sensor data is encrypted and transmitted securely. The system checks the **Message Authentication Code (MAC)** to ensure the data hasn't been altered. Once verified, the ciphertext is decrypted to retrieve the original sensor readings. This shows the system's baseline secure workflow.

***2.*Tampering Attack:**

Input: Altered Ciphertext

System Response: MAC verification failed, Data rejected

Explanation: If an attacker modifies the encrypted data during transmission, the MAC check will fail. Since the MAC is generated using the secret key, any change in ciphertext makes the MAC invalid.

3.Key Isolation:

Input: Master Key Read Attempt

System Response: Kept isolated inside ATECC608A

Explanation: The **ATECC608A cryptographic co-processor** ensures that the master key used for encryption/decryption never leaves the secure hardware. Even if an attacker tries to read or extract the key, the chip isolates it internally. This protects against key leakage and guarantees that cryptographic operations happen only inside the secure element.

4.Replay Attack:

Input: Duplicate IV Transmitted System

Response: Attack prevented via unique IV

Explanation: A replay attack occurs when an attacker resends previously captured encrypted data. To prevent this, the system uses a **unique Initialization Vector (IV)** for each encryption session. Since the IV must be fresh and non-repeating, duplicate IVs are detected and rejected, ensuring attackers cannot reuse old ciphertext to trick the system.

Conclusion And Future Scope

Conclusion:

The proposed ESP32 and ATECC608A-backed cryptographic architecture effectively resolves the most critical vulnerabilities inherent in securing IoT sensor data networks. By delegating key management to a dedicated cryptographic co-processor acting as a Hardware Root of Trust, the system completely bypasses the severe risks associated with storing static keys in standard microcontroller flash memory. The dynamic computation of an ephemeral Session Key via the ATECC608A's HMAC engine ensures Perfect Forward Secrecy for every individual transmission cycle. The implementation has proven highly resilient against diverse and sophisticated attack vectors, including eavesdropping, data modification, replay attacks, and physical memory dumping. Ultimately, this hybrid methodology delivers a robust, high performance, and highly scalable foundation for securing data integrity and confidentiality in modern IoT deployments.

FUTURE SCOPE:

The established architecture provides a robust, scalable baseline for industrial telemetry networks. Because the framework utilizes a shared I2C bus for the secure element, it naturally supports the seamless future integration of high-precision, industrial-grade I2C sensors (such as the BMP280 or BME680). This expansion can be achieved without demanding complex firmware overhauls or interfering with the established hardware Root of Trust. Future empirical research will focus on granular performance benchmarking. This includes precise oscilloscopic measurements of the AES-GCM hardware encryption latency, quantitative profiling of CPU clock cycle consumption, and exact RAM footprint tracking.